

Занятие 3: Шаблонизация и рендеринг страниц

Template engines, SSR (Server-Side Rendering)

Слайд 1: Server-Side Rendering vs Client-Side Rendering

Server-Side Rendering (SSR):

- HTML генерируется на сервере
- Браузер получает готовый HTML
- Быстрая initial load
- Лучше для SEO
- Примеры: Django, PHP, Ruby on Rails

Client-Side Rendering (CSR):

- Браузер получает JSON/JS
- HTML генерируется в браузере
- Интерактивные SPA (Single Page Applications)
- Примеры: React, Vue, Angular

SSR: Server → HTML → Browser → Display
CSR: Server → JSON → Browser → JS → HTML → Display

Слайд 2: Template Engines

Template Engine - библиотека для генерации HTML из шаблонов и данных.

Популярные template engines:

Язык	Template Engine	Синтаксис
Python	Jinja2	<code>{{ variable }}</code> , <code>{% if %}</code>
Python	Django Templates	<code>{{ variable }}</code> , <code>{% if %}</code>
Node.js	EJS	<code><%= variable %></code> , <code><% if %></code>
Node.js	Pug (Jade)	<code>#{variable}</code> , <code>if</code>
Go	Go Templates	<code>{{ .variable }}</code>

Слайд 3: Jinja2 (Python)

Установка:

```
pip install jinja2
```

Базовый пример:

```
from jinja2 import Template

template = Template('Hello {{ name }}!')
result = template.render(name='John')
# Результат: Hello John!
```

Из файла:

```
from jinja2 import Environment, FileSystemLoader

env = Environment(loader=FileSystemLoader('templates'))
template = env.get_template('index.html')
result = template.render(name='John')
```

Слайд 4: Django Templates

Настройка в settings.py:

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'],  
        'APP_DIRS': True,  
    },  
]
```

Использование в view:

```
from django.shortcuts import render  
  
def index(request):  
    context = {  
        'name': 'John',  
        'items': ['apple', 'banana', 'orange']  
    }  
    return render(request, 'index.html', context)
```

Слайд 5: Синтаксис шаблонов

Переменные:

```
{{ name }}  
{{ user.email }}  
{{ product.price|currency }}
```

Условия:

```
{% if user.is_authenticated %}  
    <p>Welcome, {{ user.name }}!</p>  
{% else %}  
    <p>Please log in</p>  
{% endif %}
```

Циклы:

```
{% for item in items %}  
    <li>{{ item }}</li>  
{% endfor %}
```

Слайд 6: Фильтры в шаблонах

Фильтры - функции для обработки переменных.

Jinja2 / Django:

```
{{ name|upper }}           # JOHN
{{ name|lower }}           # john
{{ text|truncatewords:10 }} # обрезка до 10 слов
{{ price|floatformat:2 }}  # 123.45
{{ date|date:"Y-m-d" }}    # 2024-01-01
{{ list|length }}          # длина списка
```

Цепочки фильтров:

```
{{ name|upper|truncatewords:1 }}
```

Слайд 7: Наследование шаблонов

Базовый шаблон (base.html):

```
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}My Site{% endblock %}</title>
</head>
<body>
  <header>{% block header %}{% endblock %}</header>
  <main>
    {% block content %}{% endblock %}
  </main>
  <footer>{% block footer %}{% endblock %}</footer>
</body>
</html>
```

Дочерний шаблон (index.html):

```
{% extends "base.html" %}

{% block title %}Home Page{% endblock %}

{% block content %}
```

Слайд 8: Include и компоненты

Include - включение другого шаблона:

```
{% include "header.html" %}  
{% include "footer.html" %}
```

Компоненты с параметрами:

```
{% include "card.html" with title="Hello" body="World" %}
```

card.html:

```
<div class="card">  
  <h2>{{ title }}</h2>  
  <p>{{ body }}</p>  
</div>
```

Слайд 9: Передача данных в шаблон

Простой контекст:

```
context = {  
    'name': 'John',  
    'age': 30,  
    'items': ['a', 'b', 'c']  
}  
return render(request, 'index.html', context)
```

Сложные структуры:

```
context = {  
    'user': {  
        'name': 'John',  
        'email': 'john@example.com'  
    },  
    'posts': [  
        {'title': 'Post 1', 'author': 'John'},  
        {'title': 'Post 2', 'author': 'Jane'}  
    ]  
}
```

Слайд 10: Статические файлы

Настройка в settings.py:

```
STATIC_URL = '/static/'  
STATICFILES_DIRS = [BASE_DIR / 'static']
```

Структура:

```
static/  
├── css/  
│   └── style.css  
├── js/  
│   └── main.js  
└── images/  
    └── logo.png
```

В шаблоне:

```
{% load static %}  
<link rel="stylesheet" href="{% static 'css/style.css' %}">  
<script src="{% static 'js/main.js' %}"></script>  

```

Слайд 11: EJS (Node.js)

Установка:

```
npm install ejs
```

Настройка Express:

```
app.set('view engine', 'ejs');  
app.set('views', './views');
```

Использование:

```
app.get('/', (req, res) => {  
  res.render('index', {  
    name: 'John',  
    items: ['apple', 'banana']  
  });  
});
```

Слайд 12: Синтаксис EJS

Переменные:

```
<%= name %>  
<%= user.email %>
```

Код JavaScript:

```
<% if (user) { %>  
    <p>Welcome, <%= user.name %></p>  
<% } %>  
  
<% items.forEach(function(item) { %>  
    <li><%= item %></li>  
<% }); %>
```

Include:

```
<%- include('header') %>  
<%- include('footer') %>
```

Слайд 13: Практическое задание

Задание 1: Базовый шаблон

- Создать base.html с header, main, footer
- Реализовать блоки для title, content

Задание 2: Страница списка

- Создать index.html, наследующий base.html
- Отобразить список товаров в таблице

Задание 3: Страница детализации

- Создать detail.html для одного товара
- Показать всю информацию о товаре

Задание 4: Форма

- Создать форму создания товара
- Добавить стили через static files

Слайд 14: Домашнее задание

1. Создать полноценный HTML-шаблон для блога:
 - base.html с навигацией и футером
 - index.html - список постов
 - post.html - детальная страница поста
 - create.html - форма создания поста
2. Добавить стили (CSS) через static files
3. Реализовать пагинацию на странице списка
4. Добавить фильтры в шаблоне (дата форматирование)
5. Создать README с описанием структуры шаблонов

Слайд 15: Полезные ресурсы

Документация:

- Django Templates: <https://docs.djangoproject.com/en/4.2/topics/templates/>
- Jinja2: <https://jinja.palletsprojects.com/>
- EJS: <https://ejs.co/>

Best Practices:

- Минимизировать логику в шаблонах
- Использовать наследование для DRY
- Хранить стили и скрипты в static
- Использовать фильтры для форматирования